Cloud Run Service Runbook

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This runbook is the prescriptive guide for running our Python/FastAPI services (api-service, auth-service, gateway, scheduler) on Cloud Run. It concentrates on the two scaling behaviors that most often surprise teams: cold starts when a service scales to zero, and throttling when the max-instances cap is hit during a spike. Use it whenever you deploy a latency-sensitive service or tune its autoscaling.

Standards

Cold starts and minimum instances

Cloud Run scales to zero by default. The first request after idle then pays a cold start: container boot plus every import and connection made at module load. For latency-sensitive services set `--min-instances` to at least 1 so a warm instance is always available, and keep startup cheap.

```
gcloud run deploy api-service \
 --min-instances=1 \
 --cpu-boost \
 --concurrency=80 \
 --max-instances=50
```

Trim startup work: lazy-load heavy dependencies inside the functions that need them instead of at import time, and defer building large in-memory structures until first use.

```
# Lazy import keeps cold start cheap
def get_client():
   global _client
   if _client is None:
       from heavy_sdk import Client  # imported on first use, not at boot
       _client = Client()
   return _client
```

Enable startup CPU boost so the container gets extra CPU during initialization, shortening the cold-start window.

Max instances and concurrency

Two knobs govern throughput: per-instance `--concurrency` (how many requests one instance serves at once) and `--max-instances` (the ceiling on instances). When the cap is reached under a spike, new requests queue and can time out. Size both from load tests, and leave autoscaling headroom above expected peak.

```
# Load test, then raise the ceiling (within project quota)
gcloud run services update api-service --max-instances=100
# Effective capacity ~= max-instances * concurrency
```

Set a per-instance concurrency that matches how the app uses CPU and I/O: async, I/O-bound handlers tolerate higher concurrency; CPU-heavy handlers need lower concurrency so requests are not starved.

Startup and health

Bind to `0.0.0.0:$PORT` (Cloud Run injects `$PORT`). Keep the startup path fast enough to pass the container's startup probe, and validate required configuration at boot with a clear error rather than failing deep in a request.

Common failures and fixes

- First requests after idle are multi-second slow → the service scaled to zero and cold-started, running heavy imports at startup; set `--min-instances>=1`, trim startup work and lazy-load heavy deps, and enable startup CPU boost (BUG-1044).
- Under a traffic spike requests queue and time out → the max-instances cap is reached and Cloud Run throttles; raise `--max-instances` within quota, tune per-instance concurrency, and load-test to size the limits with headroom (BUG-1046).
- Container never becomes ready on deploy → app not listening on `0.0.0.0:$PORT` or startup too slow; bind the injected port and defer heavy initialization.

Checklist

- `--min-instances>=1` for latency-sensitive services.
- Startup CPU boost enabled; heavy imports lazy-loaded, not run at module load.
- `--concurrency` tuned to the handler's CPU/I/O profile.
- `--max-instances` sized from a load test, with headroom above peak and within quota.
- App binds `0.0.0.0:$PORT`; required config validated at startup.
- Latency and instance-count metrics monitored after each deploy.

Related incidents

BUG-1044, BUG-1046